

Attention Is All You Need

Vaswani et al., Google Brain/Research, NeurIPS 2017

논문 리뷰

이 논문 전까지 시퀀스 모델링은 순서 정보에는 '순차 연산'이 필요했지만, transformer로 이 전제를 깼(시퀀스 모델링 $\neq$ 순차 연산). positional encoding으로 순서 정보를 입력에 심어두고 계산을 병렬화한 것이 그 방법.

성능 개선, 학습 비용 감소. $\rightarrow$ 구조의 변화로 성능과 효율을 함께 개선함.

번역 태스크가 주였지만, 구문 분석 실험을 통해 일반화에도 강한 아키텍처라는 점을 증명함. transformer가 단순한 '번역 모델'이 아니라 '시퀀스를 다루는 범용 구조'임을 보여줌. 또한 해석 가능성도 시사함.(attention head가 문법적, 의미적 패턴을 자발적으로 학습 가능)

▼ Abstract

기존에 존재하던 주류 시퀀스 변환 모델은 인코더, 디코더를 포함한 복잡한 recurrent or convolutional 신경망 기반, 인코더와 디코더를 attention으로 연결함.

recurrent/convolution을 배제하고 attention 매커니즘에만 기반한 새로운 아키텍처 'transformer' 제안. transformer는 번역 실험에서 뛰어난 품질, 더 좋은 병렬화, 더 짧은 학습 시간을 보임. 대규모/제한적 학습 데이터에서 구문 분석을 성공적으로 수행하면서 이 모델이 다른 과제에도 일반화 성능이 좋다는 것을 보여줌.

▼ 1. Introduction

기존에 RNN, LSTM, Gated Recurrent 신경망은 시퀀스 모델링/번역에서 확고하게 자리를 잡아옴. 관련하여 recurrent 모델, 인코더-디코더 아키텍처 연구가 이어져왔음.

recurrent 모델은 입력과 출력 시퀀스의 심볼 위치를 따라 계산하기 때문에, 위치를 계산 시점의 스텝에 맞추면서 이전 은닉 상태 h_{t-1} 과 위치 t 에 대한 시퀀스 h_t 를 생성함. $\rightarrow$ 그러나 이는 시퀀스 길이만큼 순차적으로 진행되어야 되기 때문에 병렬화가 불가능하고, 긴 시퀀스일수록 메모리 제약으로 인해서 배치 크기도 제한됨. 계산 효율을 개선해도 '순차 계산'이라는 근본적 제약이 존재.

여기서 입력이나 출력 시퀀스에서의 거리와 무관하게 의존성을 모델링하게 해주는 attention을 이용하여, transformer를 제안함. 원래는 이 attention은 RNN과 함께 사

용되었는데, 이 연구에서는 recurrence를 배제하고 attention 매커니즘만으로 인코더-디코더를 구성함. 결과적으로 성능과 학습 효율을 동시에 잡음.

▼ 2. Background

기존에 병렬화를 시도한 연구로 Extended Neural GPU, ByteNet, ConvS2S가 있는데, 모두 CNN을 기본 블록으로 써서 위치별 은닉 표현을 병렬로 계산함. 근데 이 모델들에서는 임의의 두 입출력 위치 사이의 신호를 연결하는데 필요한 연산 수가 위치 사이의 거리에 따라 증가했음. (ConvS2S는 선형, ByteNet은 로그형) → 먼 거리에서는 의존성 학습이 어려움

그러나 transformer에서는 이를 상수 개의 연산으로 줄임. 여기서 attention-weighted position을 평균내는 과정에서 유효 해상도가 떨어지는 문제가 있었으나, 이는 multi-head attention으로 상쇄함(3.2절)

이 논문의 저자는 transformer는 시퀀스 정렬 RNN이나 컨볼루션을 전혀 사용하지 않고, 오직 self-attention에만 의존해서 입출력의 표현을 계산하는 첫번째 모델에 기여했다는 것.

- self-attention(intra-attention)은 하나의 시퀀스 내의 서로 다른 위치들을 연관지어서 시퀀스 표현을 계산하는 attention 매커니즘임. 이는 독해, 요약, 합의, 문장 표현 학습 등에서 성공적으로 사용되어온 개념임.
- end-to-end memory network는 시퀀스 정렬 recurrence 대신 recurrent attention 매커니즘에 기반하고, 언어 질의응답 및 모델링에서 좋은 성능을 보임.

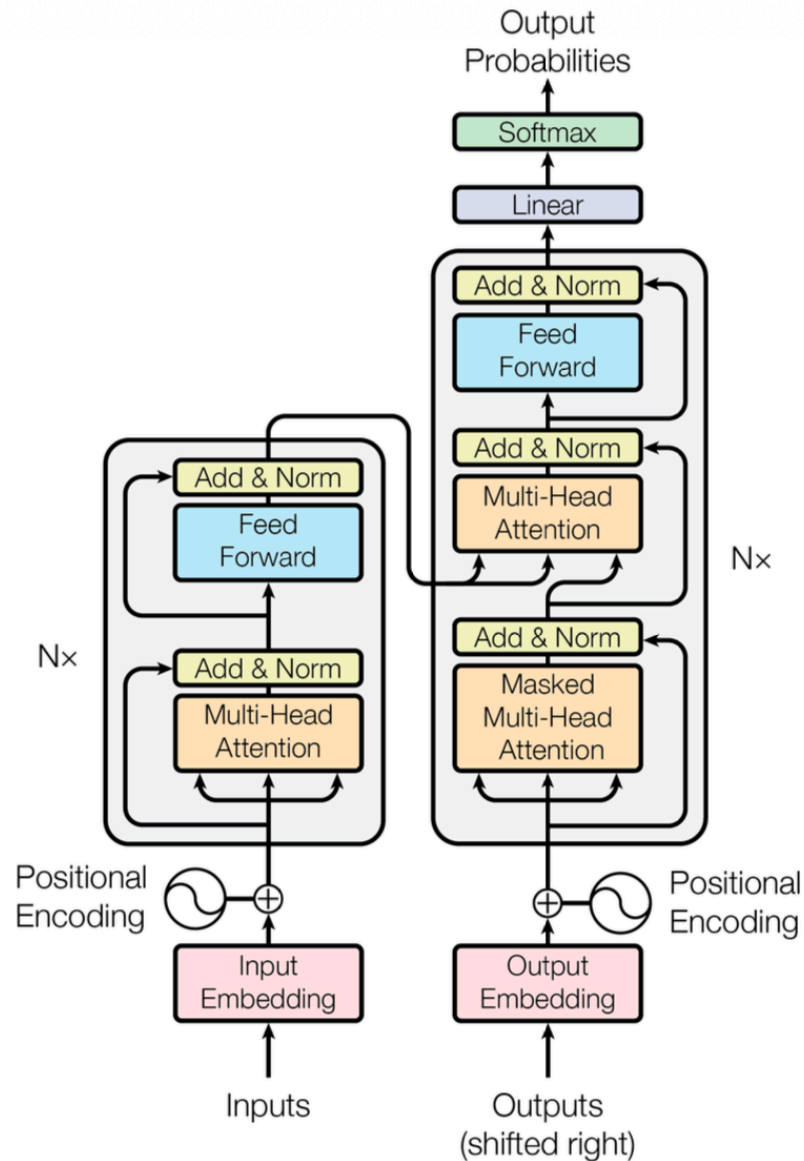
▼ 3. Model Architecture

▼ 3.1. Encoder and Decoder Stacks

시퀀스 변환 모델들은 인코더-디코더 구조를 가짐. 각 단계에서 모델은 auto-regressive(자기회귀적)이며, 다음 심볼을 생성할 때는 이전에 생성된 심볼을 추가적인 입력으로 사용함.

- 인코더: 심볼 표현의 입력 시퀀스를 연속 표현 시퀀스로 매핑
- 디코더: 주어진 연속 표현에 대해 한 원소씩 출력 시퀀스를 생성함.

transformer는 인코더, 디코더에 대한 self-attention과 point-wise한 완전 연결 레이어를 사용하는 전반적인 구조를 따름.



- 인코더

$N=6$ 개의 동일한 레이어 스택. 각 레이어는 multi-head self-attention 매커니즘, position-wise feed-forward network 이렇게 해서 두 개의 서브레이어로 구성됨.

이 두 레이어 각각에 residual connection을 적용하고, 그 뒤에 layer normalization을 적용함. → 각 서브레이어의 출력은 아래와 같음.

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

이것이 가능하려면 모든 서브레이어와 임베딩의 출력 차원이 같아야 하므로 $d_{\text{model}} = 512$ 차원으로 고정함.

- 디코더

N=6개의 동일한 레이어 스택으로 구성됨. 인코더의 두 서브레이어에 더해, 인코더 스택의 출력에 대해 multi-head attention을 수행하는 세 번째 서브레이어를 삽입함.(encoder-decoder attention)

인코더와 마찬가지로 각 서브레이어에 residual connection을 두르고, layer normalization을 적용함.

디코더의 self-attention 서브레이어는 이후 미래의 위치를 보지 못하도록 마스크됨. → 이것과 출력 임베딩이 한 칸 밀려있다는 사실과 결합해서, 위치 i에 대한 예측이 i 전의 위치 출력에만 의존하도록 보장함.

▼ 3.2 Attention

▼ 3.2.1 Scaled Dot-Product Attention

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

입력은 차원 d_k 의 Q, K, 차원 d_v 의 V로 구성됨. Q(query)와 K(key)의 내적을 계산하고, 각각을 $\sqrt{d_k}$ 로 나눈 후 softmax 함수를 적용 → V(value)에 대한 가중치 획득.

가장 많이 쓰이는 2개의 attention 함수는

- additive attention: 은닉층 하나를 가진 feed-forward network로 호환성 함수를 계산
- dot-product attention: $1/\sqrt{d_k}$ 를 제외하면 저자들의 알고리즘과 동일함. 매우 최적화된 행렬 곱셈 코드로 구현 가능하여 시간/공간 측면에서 효율적임.

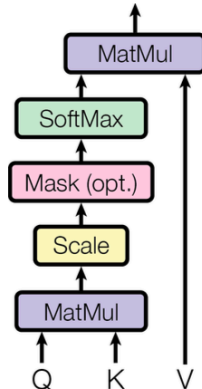
d_k 의 값이 작을 때는 두 함수가 비슷하게 동작하지만, 스케일링이 없으면 additive가 dot product보다 성능이 우수함. (d_k 값이 커질수록 내적이 커져서 softmax 함수의 gradient가 매우 작게 되는 영역으로 밀어넣을 것이라고 추정) → 이를 상쇄하기 위해 $1/\sqrt{d_k}$ 로 스케일링함.

▼ 3.2.2 Multi-Head Attention

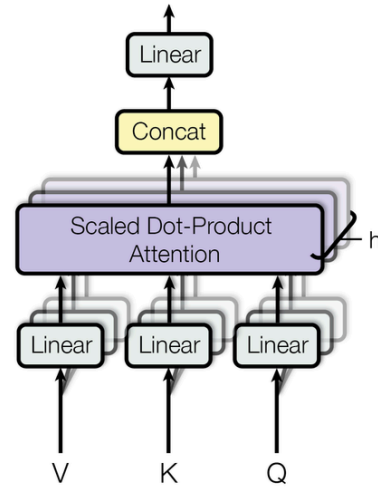
$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

```
head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)
```

Scaled Dot-Product Attention



Multi-Head Attention



d_{model} 차원의 key, value, query로 단일 attention을 수행하는 대신, 서로 다르게 학습된 선형 투영을 이용해 query, key, value를 d_k, d_k, d_v 차원으로 h 번 투영하는 것이 유익함.

- 단일 attention head의 경우 averaging 문제 발생(여러 관계를 하나의 가중 평균으로 뭉뚱그림)
- 투영된 query, key, value 각 버전에 대해 attention을 병렬로 수행하면 d_v 차원의 output value들이 만들어지는데, 이것들을 연결하고 다시 투영하면 최종값을 얻음. (multi-head attention은 모델이 서로 다른 위치에서 서로 다른 표현 부분공간의 정보에 병렬로 attend할 수 있게 해줌.) → 각 head의 차원이 줄어들어서 총 계산 비용은 전체 차원을 쓰는 단일 attention과 유사함.
 - head 수를 1, 4, 16, 32로 바꿔가며 실험한 결과, 단일 head는 최적 설정보다 BLEU가 0.9 낮고, head 수를 너무 늘려도(32) 품질이 떨어짐을 확인함.

▼ 3.2.3 Applications of Attention in our Model

transformer는 multi-head attention을 아래 세 가지 방식으로 사용함.

- **encoder-decoder attention:** Q는 이전 디코더 레이어에서, K/V는 인코더의 출력에서 옴. 디코더의 모든 위치가 입력 시퀀스 전체를 attend할 수 있

게 함. (전형적인 seq2seq 모델의 encoder-decoder attention 매커니즘을 모방함.)

- **인코더 self-attention**: self-attention 레이어에서 Q, K, V 모두 인코더 이전 레이어의 출력에서 옴. 인코더의 각 위치가 이전 레이어의 모든 위치를 attend 가능.
- **디코더 self-attention**: 디코더의 각 위치가 자기 위치까지의 디코더 내 위치들을 attend할 수 있게 해줌. auto-regressive 성질을 보존하기 위해, scaled dot-product attention 내부에서 미래 위치에 해당하는 softmax 입력값을 $-\infty$ 로 마스킹함.

▼ 3.3 Position-wise Feed-Forward Networks

인코더와 디코더의 각 레이어는 완전연결 feed-forward network를 포함함. 이는 각 위치에 개별적으로 동일하게 적용됨.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

ReLU와 두 개의 선형 변환으로 구성됨. 레이어마다 다른 파라미터를 사용함.

입출력 차원은 512, 내부 은닉층 레이어 차원은 2048인데, 이를 커널 크기가 1인 두 개의 컨볼루션으로도 설명가능하다고 함.

▼ 3.4 Embeddings and Softmax

입출력 토큰 임베딩을 d_{model} 차원의 벡터로 변환함. 학습된 선형 변환과 softmax 함수를 활용해서 디코더 출력을 다음 토큰의 확률로 변환함.

이 모델에서는 임베딩 레이어와 pre-softmax 선형 변환 사이에 동일한 가중치 행렬을 공유함. 그리고 임베딩 레이어에서는 이 가중치에 $\sqrt{d_{\text{model}}}$ 을 곱함.

▼ 3.5 Positional Encoding

모델에 recurrence와 convolution이 없으므로, 순서를 활용하도록 하려면 시퀀스 내 토큰의 위치 정보를 주입할 방법이 필요함. → 인코더, 디코더 스택 맨 아래의 입력 임베딩에 positional encoding을 더함. (임베딩과 동일한 d_{model} 차원을 가지므로 더할 수 있음.)

$$\begin{aligned}\text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})\end{aligned}$$

positional encoding는 학습된 방식, 고정된 방식 등 여러 방식들을 사용할 수 있음. 이 연구에서는 서로 다른 주파수의 사인, 코사인 함수를 사용함. 함수 선택 이유는

- 임의의 고정된 offset k 에 대해서 $PE_{\{pos+k\}}$ 가 PE_{pos} 의 선형 함수로 표현될 수 있어서, 모델이 상대적 위치로 attend하는 법을 쉽게 학습할 수 있을 것.
- 학습 가능한 positional embedding과 비교했을 때 거의 동일한 성능을 보였지만, 학습 때보다 더 긴 시퀀스 길이에도 모델이 외삽할 수 있게 해줄 수 있으므로 sinusoidal 방식을 선택함.

▼ 4. Why Self-Attention

self-attention 레이어 vs recurrent / convolutional 레이어(전형적인 시퀀스 변환 인코더/디코더의 은닉층)를 비교하기 위한 세 가지 지표

- 레이어당 총 계산 복잡도
- 병렬화될 수 있는 계산량 (순차 연산 최소 수)
- 네트워크 내 장거리 의존성 사이의 경로 길이(의존성 학습 능력에 영향을 미치는 핵심 요인은 forward, backward 신호가 네트워크를 통과해야하는 경로의 길이. 이 경로가 짧을 수록 장거리 의존성 학습이 쉬워짐.)

레이어	레이어당 계산 복잡도	순차 연산	최대 경로 길이
Self-Attention	$O(n^2 \cdot d)$ ** 시퀀스 길이 n 이 표현 차원 d 보다 작을 때 self-attention이 recurrent 보다 빠름	$O(1)$	$O(1)$ ** 길이가 짧을 수록 장거리 의존성 학습 이 쉬워짐.
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	인접 커널의 경우 $O(n/k)$ 개의 레이어 스택 필요 dilated인 경우 $O(\log_k(n))$ 개의 레 이어 스택 필요
Self-Attention (restricted) ** 매우 긴 시퀀스를 다루 기 위해 self-attention을 입력 시퀀스 내 크기 r 의 이웃만 보도록 제한	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$ ** 향후 탐구 계획

self-attention은 해석 가능한 모델을 만들어낼 수도 있음. 개별 attention head들은 서로 다른 과제를 수행하는 법을 명확히 학습하고, 서로 다른 head는 문장의 통사적/의미적 구조와 관련된 행동을 보이는 것으로 나타남.

▼ 5. Training / 6. Results

- 데이터/배치

- WMT2014 En-De(약 450만 문장쌍, byte-pair encoding, 공유 vocab 약 37000 토큰)
- En-Fr(3600만 문장, word-piece vocab 32000 토큰)
- 문장쌍은 시퀀스 길이 기준으로 배치화하여 배치당 약 25000 소스 토큰 + 25000 타겟 토큰의 문장 쌍 집합으로 구성

- **Optimizer:** Adam ($\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=1e-9$)

- **Learning rate schedule**

```
lr = d_model-0.5 · min(step-0.5, step · warmup_steps-1.5)
```

처음 warmup_steps(=4000) 동안 학습률을 선형적으로 증가시키고, 이후 step number의 역제곱근에 비례해 감소시킴.

- **Residual Dropout**

- 각 서브레이어의 출력이 residual에 더해지기 전
- 인코더, 디코더 스택 모두에서 임베딩과 positional encoding의 합
- base 모델은 $P_{drop}=0.1$

- **Label Smoothing:** $\epsilon_{ls}=0.1$ (모델이 불확실해지게 학습되어서 perplexity는 손해지만, accuracy와 BLEU는 개선됨.)

아래는 결과

- **EN-DE**

- big transformer가 기존 최고 모델(양상블 포함)을 2.0 BLEU 이상 능가(EN-DE BLEU 28.4)
- base transformer 조차도 기존 모든 모델과 양상블 능가, 학습 비용도 가장 적음.

- **EN-FR**

- big transformer가 기존 단일 모델 능가하는 41.8 BLEU 달성. 이전 최고 모델 학습 비용의 1/4 미만 사용.

- 디코딩은 beam size 4, length penalty $\alpha=0.6$ 인 beam search를 사용했고, base 모델은 마지막 5개, big 모델은 마지막 20개 체크포인트를 평균 내어 최종 모델로 씬. 출력 최대 길이는 입력 길이+50으로 제한하되 가능하면 조기 종료함.

- **model variations**

- head 수 1개(단일 head)는 최적 설정보다 BLEU 0.9 낮고, head 수를 너무 늘려도(32) 품질 저하
- attention key 크기 d_k 를 줄이면 품질이 나빠짐 → 호환성을 결정하는 게 쉽지 않고, dot product보다 더 정교한 호환성 함수가 유익할 수 있음.
- 모델 크기를 키우면 성능이 좋아짐.
- dropout이 과적합 방지에 도움됨.
- sinusoidal positional embedding을 학습된 positional embedding으로 바꿔도 sinusoidal 방식과 거의 동일한 결과

번역 이외의 task에도 일반화되는지 확인하기 위해 영어 구문 분석 실험을 함. (출력이 강한 구조적 제약, 입력보다 상당히 길다는 점에서 어려운 과제. RNN seq2seq는 소규모 데이터 환경에서 좋은 성능 달성 불가.)

Penn Treebank의 Wall Street Journal 부분(약 4만개 문장)으로 4-layer transformer 학습, 1700만 문장으로 구성된 고신뢰도, 준지도 환경에서도 학습. → 하이퍼파라미터는 EN-DE 번역 base 모델에서 거의 바꾸지 않았는데도, WSJ 단독 학습에서 F1 91.3, 준지도 학습(약 1700만 문장 추가)에서 F1 92.7을 기록함.

RNN seq2seq 모델들이 소량 데이터에서 최고 성능을 내지 못했던 것과 달리 transformer는 task-specific 튜닝 없이도 좋은 성능을 보임.

▼ 7. Conclusion

transformer = recurrence 대신 multi-headed self-attention으로 인코더-디코더의 recurrent layer를 대체한 최초의 시퀀스 변환 모델

번역 태스크에서 RNN/CNN 기반 구조보다 훨씬 빠르게 학습됨. EN-DE에서는 최고 모델이 기존 보고된 모든 앙상블 모델을 능가함.

향후 연구 방향

- 텍스트 이외의 입출력 모달리티(이미지, 오디오, 비디오)로 확장
- 큰 입출력을 효율적으로 다루기 위한 local/restricted attention 메커니즘 연구
- 생성 과정을 덜 순차적으로 만드는 것.